

SVG Model

A Nasdanika Zephyr Book

Pavel Vlasov

Version 2026.11.0, 2026-11-07

Table of Contents

Colophon . . . 1

Preface . . . 2

Who this book is for . . . 2

A book that stands on its own . . . 2

What you need . . . 2

Conventions . . . 3

I: The Why . . . 4

1. Generated diagrams . . . 5

1.1. Who lays it out . . . 5

1.2. Why not generate PlantUML . . . 5

1.3. Why SVG . . . 6

1.4. Who measures the text . . . 6

2. SVG as a model . . . 7

2.1. Why a model and not strings . . . 7

2.2. A pragmatic subset . . . 7

2.3. Written by EMF, not by hand . . . 7

2.4. Links and tooltips . . . 8

II: The How . . . 9

3. Getting started . . . 10

3.1. Adding the model . . . 10

3.2. Writing and reading without a resource set . . . 10

3.3. In a resource set . . . 11

3.4. What the build does . . . 11

4. The model . . . 12

4.1. Classes and elements . . . 12

4.2. The base classes . . . 13

4.3. Children and text . . . 13

5. Building, writing and reading . . . 15

5.1. Building and writing . . . 15

5.2. Reading . . . 16

5.3. In a resource set . . . 16

6. From a diagram to SVG . . . 18

6.1. The diagram . . . 18

6.2. Laying out and drawing . . . 18

6.3. The converter . . . 19
III: Looking Ahead . . . **24**
7. Looking ahead . . . **25**
7.1. A diagram to SVG module . . . 25
7.2. Measured text . . . 25
7.3. Class context diagrams . . . 25
7.4. Dark mode . . . 25
7.5. PNG . . . 25
7.6. Reading other tools' SVG losslessly . . . 26
Appendix A: Reference . . . **27**
A.1. Java classes . . . 27
A.2. Writing options . . . 27
Appendix B: Building this book, and its figure . . . **28**
B.1. The figure . . . 28
Appendix C: Glossary . . . **29**

Colophon

SVG Model: A Nasdanika Zephyr Book

Edition 2026.11.0, built on 2026-11-07.

This edition describes the SVG model of the Nasdanika SDK Runtime 2026.11.0, Maven coordinates `org.nasdanika.sdk.runtime.models.svg:model:2026.11.0`, namespace URI `www.w3.org/2000/svg`. Every listing in it is included from the module's tests, which are compiled and run when the book is built. Its figure was generated by one of those tests: a diagram laid out with ELK and written as SVG with the model.

Copyright © 2026 Pavel Vlasov.

The text and figures of this book are licensed under the Creative Commons Attribution 4.0 International License (CC BY 4.0). You may share and adapt them for any purpose, commercial purposes included, provided you give appropriate credit, provide a link to the license, and indicate if you made changes. A suggested credit: "Pavel Vlasov, *SVG Model: A Nasdanika Zephyr Book*, licensed under CC BY 4.0", with a link to github.com/Nasdanika-SDK/runtime.

The code is licensed under the Eclipse Public License 2.0, the license of the runtime. In addition, you may use the code of this book's listings in your own programs and documentation without asking for permission and without attribution.

Neither license covers the Nasdanika name or logo: CC BY 4.0 grants no trademark rights.

Eclipse, EMF and ELK are trademarks of the Eclipse Foundation. SVG is a trademark of the World Wide Web Consortium. draw.io is a trademark of JGraph Ltd. Java is a trademark of Oracle and its affiliates. Other names may be trademarks of their owners. Their use here identifies the technologies the model relates to and implies no endorsement.

The source of this book, its tests and the model it describes are at github.com/Nasdanika-SDK/runtime, in `models/svg/model`.

Typeset with Asciidoctor PDF in Noto Sans and M+ 1mn, in the look of the Nasdanika Zephyr Books.

Preface

This book is about SVG as an EMF model: a subset of SVG, the twenty-odd elements and the presentation attributes that generated diagrams need, as Java classes that EMF writes as SVG and reads back. Its point is generated diagrams. A model is turned into a diagram, ELK lays the diagram out, and the SVG model draws it, with every shape a link and every documented shape a tooltip.

The first part of the book is the why: why generated diagrams should be SVG, and why SVG should be a model. The second part is the how, for Java developers, and ends with a diagram laid out with ELK and written as SVG, which is this book's figure. The third says what comes next.

Who this book is for

Developers who generate diagrams from models for documentation, sites and books, and want them in vector form, navigable, and the same in HTML and PDF.

Writers of technical documentation who want to know why a diagram in a Nasdanika book looks as it does, and how to get the same in their own.

A book that stands on its own

This book is one of the Nasdanika Zephyr Books: in-depth manuals, one per module of the Nasdanika SDK Runtime, in the manner of IBM Redbooks. Two of them are close to this one: *Diagram Model*, the notation-neutral model of diagrams, and *Diagram Layout with ELK*, which lays it out. You do not need either to read this one: where the books touch, this one repeats what you need.

The series is named after the Bootswatch theme it is styled with, Zephyr, and has nothing to do with the Zephyr real-time operating system or with SmartBear's Zephyr test management.

What you need

- **A JDK for Java 25** and **Maven 3.9.16 or later**.
- The model's Java module is `org.nasdanika.sdk.runtime.models.svg`, its Maven coordinates are

`org.nasdanika.sdk.runtime.models.svg:model:2026.11.0`, and its namespace URI is SVG's own, `www.w3.org/2000/svg`.

- Some familiarity with EMF helps in part two: an EMF model is a set of Java interfaces generated from a metamodel, with containment, references, reflection and persistence for free.

Conventions

- **Every listing is included from a test** of the module and compiled and run when the book is built. Callouts, such as <1>, explain the lines they mark.
- **The figure is generated.** It is the SVG file a test writes, included as it is, in vector form, in the HTML, the PDF and the EPUB.
- **Status labels** mark what this edition does not have yet: PLANNED for designed but not built, SKETCH for partly built.
- **Voice.** I write in the first person.

Part I: The Why

Why generated diagrams should be SVG, and why SVG should be a model. No code is needed to read this part.

Chapter 1. Generated diagrams

Many diagrams in documentation are not drawn by anyone. They are generated: the classes of a model and how they relate, the modules of a system and what each requires, the systems of an architecture and how they talk. A person would draw them worse, more slowly, and once, while the model they show keeps changing.

1.1. Who lays it out

The deciding question for a diagram is who arranges it. The runtime has three answers, each with one job:

Who lays it out	Notation	Path	Merge needed
A layout engine, from a model	Boxes and lines: context views, module graphs, class diagrams	Diagram model, ELK, SVG model, SVG	None: it is regenerated, never edited
A person	Anything	Draw.io, as an editable PNG	By id, when a generated diagram is arranged by hand
A person, in text	UML written by hand	PlantUML, through Asciidoctor Diagram	None

The first row is the cheapest, and this book is about its last step. Generated SVG needs neither an external notation nor a merge, so most generated diagrams should take it. Draw.io stays for diagrams that someone wants to arrange.

1.2. Why not generate PlantUML

For years I generated PlantUML text from models and let PlantUML draw it. That is a translation that loses what matters: a class in generated PlantUML text no longer knows the class of the model it came from, so it cannot link to its documentation or show it as a tooltip without more encoded text. And generating text only for it to be parsed and drawn is a detour.

PlantUML remains the right tool for diagrams that people write by hand. For generated ones, the

diagram model and ELK do what PlantUML did here, and keep the link to the model.

1.3. Why SVG

- **Vector.** It is sharp at any size, in print and on high-density screens.
- **Everywhere.** Browsers draw it, Asciidoctor PDF draws it into PDF through prawn-svg, and EPUB readers draw it. Only some platforms, Medium and LinkedIn among them, want PNG, and for those a browser can turn SVG into PNG.
- **Navigable.** Every shape can be a link, to the page of the element it depicts, and have a title, which browsers show as a tooltip. A generated diagram in an HTML book or a site is a map of the documentation.
- **Text.** It is diffable, so a change to a generated diagram shows in a review.

1.4. Who measures the text

ELK needs the size of every label before it can lay a diagram out, and the SVG must draw the text in the font that was measured. This edition estimates text with an average character width, which needs no fonts and is the same on every machine, but which is a little off for proportional fonts. A table of glyph widths for the font the books use, shipped as a resource, is PLANNED: accurate, still the same on every machine, and with no dependency on the fonts installed.

Chapter 2. SVG as a model

2.1. Why a model and not strings

SVG is XML, so it can be written with string templates or with a DOM. Both work for a first diagram. Neither knows SVG: a misspelled attribute is written as it is, a child is put where it does not belong, and every generator repeats the same escaping and formatting.

A model gives each element a class and each attribute a typed feature. A generator says `rect.setRx(6)`, the compiler checks it, and the model writes it. It also gives SVG what every EMF model has: a resource set, reflection, copying, comparison, and a place in a pipeline of models, between the diagram model and the file.

2.2. A pragmatic subset

SVG 1.1 has some sixty elements, and its weight is in the attributes: dozens of presentation attributes, plus CSS. Diagrams need a fraction:

- **Structure:** `svg`, `g`, `defs`, `a`, `marker`, `clipPath`, `linearGradient` and `stop`.
- **Shapes:** `rect`, `circle`, `ellipse`, `line`, `polyline`, `polygon`, `path`, `image` and `use`.
- **Text:** `text`, `tspan`, `title`, `desc` and `style`.
- **Presentation attributes:** fill and stroke with their opacity, width, dashes, caps and joins; opacity; the font attributes; text anchor and baseline; markers; visibility and clipping.
- **On every element:** `id`, `class`, `style` and `transform`.

The subset is set by what generated diagrams need and by what Asciidoctor PDF's SVG renderer, prawn-svg, draws. prawn-svg draws the basic shapes, paths, text, `use`, markers and linear gradients, and not `foreignObject`, filters or masks. Staying inside it means that one SVG works in the HTML, the PDF and the EPUB of a book; this book's figure is the test.

2.3. Written by EMF, not by hand

SVG has no official XML schema to generate a model from, so the model is written by hand, in Xcore. Its classes and features are annotated with ExtendedMetaData, the annotations EMF puts on models it generates from XML schemas: this class is the element `rect`, this feature is the

attribute `stroke-width`. EMF's XML resource reads the annotations, so it writes SVG and reads it back, and there is no serializer to write.

Two consequences shape the model:

- **The namespace URI is SVG's.** An EMF package's namespace URI is the XML namespace its elements are written in, so this package's is `www.w3.org/2000/svg`, not a runtime.sdk.nasdanika.org URI like the runtime's other models.
- **Children are in a feature map.** A group holds rectangles, circles, texts and groups in one sequence, and the order is the drawing order. A typed list per kind of child would lose it. So a container keeps its children in a feature map keyed by element name, and text keeps its characters and spans in another. The model's operations `add`, `children`, `addText`, `addSpan` and `text` hide the feature maps.

2.4. Links and tooltips

Every shape a generator draws stands for an element of a model. Wrapped in an `a`, the shape links to that element's page. Given a `title`, it shows the element's documentation as a tooltip. That is the difference between a generated diagram and a picture of one, and why the subset has both.

Part II: The How

The model in Java: adding it to a project, its classes, building, writing and reading SVG, and drawing a diagram that ELK has laid out.

Chapter 3. Getting started

The model is an EMF model, defined in Xcore and generated into Java when the runtime is built.

3.1. Adding the model

The runtime's models have a bill of materials, `org.nasdanika.sdk.runtime.models:bom`, which manages their versions. Import it in `dependencyManagement`, and depend on `org.nasdanika.sdk.runtime.models.svg:model` without a version.

The model is a named Java module:

Listing 1. The module declaration of the SVG model

```
module org.nasdanika.sdk.runtime.models.svg {

    exports org.nasdanika.sdk.runtime.models.svg;
    exports org.nasdanika.sdk.runtime.models.svg.impl;
    exports org.nasdanika.sdk.runtime.models.svg.resource;
    exports org.nasdanika.sdk.runtime.models.svg.services;
    exports org.nasdanika.sdk.runtime.models.svg.util;

    requires transitive org.nasdanika.sdk.runtime.common;
    requires transitive org.eclipse.emf.ecore.xmi;

    provides EPackage with SvgEPackageProvider;

    provides CapabilityFactory with SvgCapabilityFactory;

    provides ResourceSetContributor with SvgResourceSetContributor;

}
```

It requires the runtime's `common` module transitively, which brings EMF with it, and EMF's XML resource. It provides its `EPackage`, a contents handler for `.svg` resources, and a resource set contributor that registers the resource factory for the extension.

3.2. Writing and reading without a resource set

`SvgFormat` writes an `svg` element to a stream or a string, and reads one from a stream. That is all a generator needs (Chapter 5).

3.3. In a resource set

`NasdanikaResourceSet.createAndConfigure()` creates a resource set and lets every module on the module path contribute to it. With the model on the module path, a `.svg` file loads as its root `Svg` element, and a resource with one saves as SVG.

The resource factory reads the dots in a file name as qualifiers, right to left, and asks for a contents handler for each, starting with the leftmost. The SVG model's handler answers for the last qualifier, `svg`, so a more specific one wins for a name with more: `system.drawio.svg` loads as a Draw.io document when the runtime's Draw.io model is on the module path, from the document that Draw.io embeds in its SVG export.

3.4. What the build does

The Xcore definition is in `src/main/resources/org/nasdanika/sdk/runtime/models/svg/svg.xcore`. Maven generates the Java sources into `src-gen` with the Xtext Maven plugin. The generated API is in three packages: the interfaces in `org.nasdanika.sdk.runtime.models.svg`, their implementations in `impl`, and the switch and adapter factory in `util`. `SvgFormat` and the contents handler are hand-written, in `resource`; the services are in `services`.

Chapter 4. The model

4.1. Classes and elements

Each class is an SVG element, named after it: `Rect` is `rect`, `G` is `g`, `TSpan` is `tspan`.

Class	Element	Its own attributes
`Svg`	`svg`	`version`, `x`, `y`, `width`, `height`, `viewBox`, `preserveAspectRatio`
`G`	`g`	
`Defs`	`defs`	
`A`	`a`	`href`, `target`
`Marker`	`marker`	`viewBox`, `refX`, `refY`, `markerWidth`, `markerHeight`, `orient`, `markerUnits`
`ClipPath`	`clipPath`	`clipPathUnits`
`LinearGradient`	`linearGradient`	`x1`, `y1`, `x2`, `y2`, `gradientUnits`
`Stop`	`stop`	`offset`, `stop-color`, `stop-opacity`
`Rect`	`rect`	`x`, `y`, `width`, `height`, `rx`, `ry`
`Circle`	`circle`	`cx`, `cy`, `r`
`Ellipse`	`ellipse`	`cx`, `cy`, `rx`, `ry`
`Line`	`line`	`x1`, `y1`, `x2`, `y2`
`Polyline`, `Polygon`	`polyline`, `polygon`	`points`
`Path`	`path`	`d`
`Image`	`image`	`href`, `x`, `y`, `width`, `height`, `preserveAspectRatio`
`Use`	`use`	`href`, `x`, `y`
`Text`, `TSpan`	`text`, `tspan`	`x`, `y`, `dx`, `dy`
`Title`, `Desc`, `Style`	`title`, `desc`, `style`	`value`, the content; `type` for `style`

Feature names are the attribute names in camel case: `stroke-width` is `strokeWidth`. Two are renamed: the `class` attribute is `styleClass`, and the `style` attribute is `styleAttribute`, because `getClass()` belongs to Java and `Style` is the element.

Coordinates and sizes of shapes are doubles. Attributes that SVG allows to carry units, percentages or lists, such as `width` of `svg`, `x` of `text`, `stroke-width` and `points`, are strings, written as given. An attribute that is not set is not written, and SVG's default applies.

4.2. The base classes

Class	What it has
`Element`, abstract	`id`, `styleClass`, `styleAttribute`, `transform`: what every element of the subset has.
`Presentation`, interface	The presentation attributes: `fill`, `fillOpacity`, `stroke`, `strokeWidth`, `strokeOpacity`, `strokeDasharray`, `strokeLinecap`, `strokeLinejoin`, `opacity`, `fontFamily`, `fontSize`, `fontWeight`, `fontStyle`, `textAnchor`, `dominantBaseline`, `markerStart`, `markerMid`, `markerEnd`, `visibility`, `clipPath`. Presentation attributes are inherited in SVG, so setting them on a group sets them for its children.
`Container`, abstract	Children, in `content`. Every structure element and every shape is a container, so a shape can have a `title`.
`Shape`, abstract	The shapes.
`TextContent`, abstract	Characters and spans, in `mixed`, and the position attributes of text.
`DocumentRoot`	The document: the root `svg` element and the namespace prefixes. `SvgFormat` creates it.

4.3. Children and text

A container's children are in a feature map, `content`, so that one sequence holds elements of every kind in drawing order. Text's characters and spans are in another, `mixed`. The model's operations work with them without the feature maps:

Operation	Does
`container.add(child)`	Appends a child and returns it, typed as it was given
`container.children()`	The children, in order
`text.addText(characters)`	Appends characters

Operation	Does
`text.addSpan(span)`	Appends a span and returns it
`text.text()`	The characters of the text and its spans, in order

Chapter 5. Building, writing and reading

5.1. Building and writing

Listing 2. A group, a rectangle with a tooltip, a circle and text (`SvgTests.java`)

```
SvgFactory factory = SvgFactory.eINSTANCE;
Svg svg = factory.createSvg();
svg.setWidth("120");
svg.setHeight("60");

G group = svg.add(factory.createG()); ①
group.setFill("#dae8fc");
group.setStroke("#6c8ebf");

Rect rect = group.add(factory.createRect());
rect.setWidth(80);
rect.setHeight(40);
rect.setRx(6);
Title title = rect.add(factory.createTitle()); ②
title.setValue("A tooltip");

Circle circle = group.add(factory.createCircle());
circle.setCx(100);
circle.setCy(20);
circle.setR(10);

Text text = svg.add(factory.createText());
text.setX("40");
text.setY("55");
text.setTextAnchor("middle");
text.addText("Hello, "); ③
TSpan bold = text.addSpan(factory.createTSpan());
bold.setFontWeight("bold");
bold.addText("SVG");

String written = SvgFormat.toString(svg); ④
```

① `add` appends a child and returns it, typed. The group's fill and stroke apply to its children.

② A title, the rectangle's tooltip in a browser.

③ Text is characters and spans, in order.

④ The SVG, written by EMF's XML resource.

What it writes:

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<svg xmlns="http://www.w3.org/2000/svg" width="120" height="60">
  <g fill="#dae8fc" stroke="#6c8ebf">
    <rect width="80.0" height="40.0" rx="6.0">
      <title>A tooltip</title>
    </rect>
    <circle cx="100.0" cy="20.0" r="10.0"/>
  </g>
  <text text-anchor="middle" x="40" y="55">Hello, <tspan font-weight="bold">SVG</tspan></text>
</svg>
```

SVG's namespace is the default one, so the elements have no prefix, and the output can also be pasted into an HTML page. Attributes come in the order of the model's features, the presentation attributes first.

5.2. Reading

Listing 3. Reading the written SVG back

```
Svg read = SvgFormat.load(new ByteArrayInputStream(written.getBytes(StandardCharsets.UTF_8)),
URI.createURI("hello.svg"));
G readGroup = (G) read.children().getFirst();
assertThat(readGroup.children()).hasSize(2).first().isInstanceOf(Rect.class);
Text readText = (Text) read.children().getLast();
assertThat(readText.text()).isEqualTo("Hello, SVG");
```

Elements and attributes outside the subset are skipped when SVG is read, so an SVG file written by another tool loads as far as the model reaches. Keeping what the model does not have, so that such a file can be changed and written back without losses, is **PLANNED**.

5.3. In a resource set

Listing 4. Saving and loading `.svg` resources

```
ResourceSet resourceSet = NasdanikaResourceSet.createAndConfigure();
File file = new File("target/svg/rect.svg");
Resource resource = resourceSet.createResource(URI.createFileURI(file.getAbsolutePath())); ①
resource.getContents().add(svg);
resource.save(null);

Resource loaded = NasdanikaResourceSet.createAndConfigure().getResource(URI.createFileURI(file
.getAbsolutePath()), true);
Svg read = (Svg) loaded.getContents().getFirst(); ②
assertThat(read.getWidth()).isEqualTo("10");
```

① A resource for a `.svg` file. Its contents is one `Svg` element.

② Loaded, the contents is the root `Svg` element.

Chapter 6. From a diagram to SVG

This chapter builds a diagram with the runtime's diagram model, lays it out with ELK, and draws it as SVG. The diagram is the system context of an Internet Banking System, after the C4 model's example.

6.1. The diagram

Listing 5. A system context in the diagram model (`SvgTests.java`)

```
static Diagram context() {
    DiagramFactory factory = DiagramFactory.eINSTANCE;
    Diagram diagram = factory.createDiagram();
    diagram.getAttributes().put("elk.direction", "DOWN"); ①
    diagram.getAttributes().put("elk.edgeRouting", "ORTHOGONAL");
    diagram.getAttributes().put("elk.spacing.nodeNode", "40");

    Node customer = node(diagram, "customer", "Personal Banking Customer", "person");
    Node banking = node(diagram, "banking", "Internet Banking System", "software-system");
    banking.setDocumentation("Lets customers view their accounts and make payments"); ②
    banking.getAttributes().put("link", "https://runtime.sdk.nasdanika.org");
    Node mainframe = node(diagram, "mainframe", "Mainframe Banking System", "external");
    Node email = node(diagram, "email", "E-mail System", "external");

    connect(diagram, customer, banking, "Views accounts and makes payments");
    connect(diagram, banking, mainframe, "Gets account information");
    connect(diagram, banking, email, "Sends e-mail").getAttributes().put("async", "true");
    connect(diagram, email, customer, "Sends e-mails to");
    return diagram;
}
```

① ELK's layout options are attributes of the diagram: top to bottom, orthogonal routes, and room between nodes.

② Documentation, which becomes a tooltip, and a link attribute, which becomes a hyperlink.

The diagram has nodes and connections, and no coordinates.

6.2. Laying out and drawing

Listing 6. Layout, conversion and writing

```
Diagram diagram = context();
new ElkLayout().padding(16, 12).layout(diagram); ①
Svg svg = new DiagramToSvg().convert(diagram); ②
```

```
Path file = Path.of("target", "svg", "internet-banking.svg");
Files.createDirectories(file.getParent());
Files.writeString(file, SvgFormat.toString(svg)); ③
```

① ELK sizes the nodes to fit their names, with padding, places them, routes the connections, and places a label for each connection's name. It writes bounds, routes and label positions into the diagram.

② The converter, below, draws the laid-out diagram with the SVG model.

③ The SVG, written to a file.

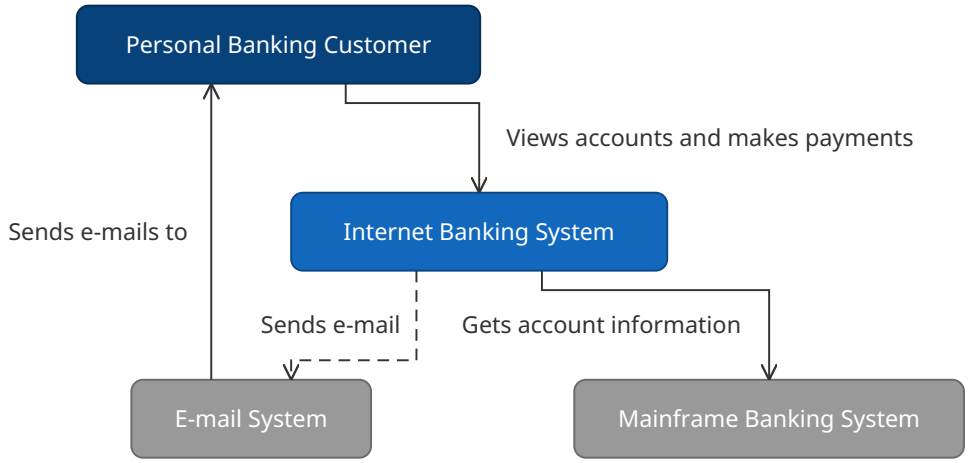


Figure 1. The system context, laid out by ELK and drawn as SVG (`internet-banking.svg`)

In the HTML edition of this book, the Internet Banking System box is a link, and its documentation is its tooltip.

6.3. The converter

The converter is in the module's tests: it is a demonstration, and a diagram's look is a choice of the generator, not of the model. It walks the diagram, keeping track of where each node's coordinates start, because the bounds of a node are relative to the node that contains it:

Listing 7. Walking the diagram (`DiagramToSvg.java`)

```
public class DiagramToSvg {

    static final String FONT = "Noto Sans, Helvetica, Arial, sans-serif";
```

```
    static final double FONT_SIZE = 12;
    static final double MARGIN = 10;

    /** Fill and stroke by the type attribute of a node */
    static final Map<String, String[]> PALETTE = Map.of(
        "person", new String[] { "#08427b", "#052e56", "#ffffff" },
        "software-system", new String[] { "#1168bd", "#0b4884", "#ffffff" },
        "external", new String[] { "#999999", "#6b6b6b", "#ffffff" });

    static final String[] DEFAULT = { "#f5f5f5", "#666666", "#333333" };

    private final SvgFactory factory = SvgFactory.eINSTANCE;

    public Svg convert(Diagram diagram) {
        Svg svg = factory.createSvg();
        double[] extent = { 0, 0 };
        measure(diagram, 0, 0, extent);
        double width = extent[0] + 2 * MARGIN;
        double height = extent[1] + 2 * MARGIN;
        svg.setWidth(number(width)); ①
        svg.setHeight(number(height));
        svg.setViewBox("0 0 " + number(width) + " " + number(height));
        svg.setFontFamily(FONT);
        svg.setFontSize(number(FONT_SIZE));
        svg.add(arrowhead());

        G content = svg.add(factory.createG()); ②
        content.setTransform("translate(" + number(MARGIN) + "," + number(MARGIN) + ")");
        draw(diagram, content, 0, 0);
        return svg;
    }

    /**
     * An open arrowhead, drawn at the end of a polyline that refers to it.
     */
    private Defs arrowhead() {
        Defs defs = factory.createDefs();
        Marker marker = defs.add(factory.createMarker());
        marker.setId("arrow");
        marker.setViewBox("0 0 10 10");
        marker.setRefX("10");
        marker.setRefY("5");
        marker.setMarkerWidth("8");
        marker.setMarkerHeight("8");
        marker.setOrient("auto");
        Path path = marker.add(factory.createPath());
        path.setD("M 0 0 L 10 5 L 0 10");
        path.setFill("none");
        path.setStroke("#333333");
        path.setStrokeWidth("1.5");
        return defs;
    }

    /**
```

```
     * Draws the children of an element. Bounds of nodes are relative to the nearest enclosing
     * node, and routes of connections are in the coordinates of the nearest enclosing node.
     */
    private void draw(DiagramElement element, Container parent, double x, double y) {
        for (DiagramElement child : element.getChildren()) {
            if (child instanceof Node node && !(child instanceof Label) && node.getBounds() !=
null) {
                Bounds b = node.getBounds();
                node(node, parent, x + b.getX(), y + b.getY());
                draw(node, parent, x + b.getX(), y + b.getY());
            } else if (child instanceof Connection connection) {
                connection(connection, parent, x, y);
            } else if (!(child instanceof Node)) {
                draw(child, parent, x, y); // Pages and layers are transparent
            }
        }
    }
```

① The size of the drawing, from the extent of the laid-out diagram, and a view box in the same units.

② A group with a margin, moved by a transform.

A node is a group with a rounded rectangle and its name, in colors by its `type` attribute:

Listing 8. A node

```
private void node(Node node, Container parent, double x, double y) {
    Container target = parent;
    String link = node.getAttributes().get("link");
    if (link != null) {
        A a = parent.add(factory.createA()); ①
        a.setHref(link);
        target = a;
    }
    G g = target.add(factory.createG());
    g.setId(node.getId());
    if (node.getDocumentation() != null) {
        Title title = g.add(factory.createTitle()); ②
        title.setValue(node.getDocumentation());
    }
    String[] colors = PALETTE.getOrDefault(node.getAttributes().get("type"), DEFAULT);
    Rect rect = g.add(factory.createRect());
    rect.setX(round(x));
    rect.setY(round(y));
    rect.setWidth(round(node.getBounds().getWidth()));
    rect.setHeight(round(node.getBounds().getHeight()));
    rect.setRx(6);
    rect.setFill(colors[0]);
    rect.setStroke(colors[1]);
    if (node.getName() != null) {
        Text text = g.add(factory.createText()); ③
```

```
        text.setX(number(x + node.getBounds().getWidth() / 2));
        text.setY(number(y + node.getBounds().getHeight() / 2 + FONT_SIZE * 0.35));
        text.setTextAnchor("middle");
        text.setFill(colors[2]);
        text.addText(node.getName());
    }
}
```

① A node with a `link` attribute is wrapped in a link.

② A node's documentation is the group's title: its tooltip.

③ The name, centered. The baseline is moved down by a third of the font size, which centers lowercase text closely enough, and works in renderers that do not support `dominant-baseline`.

A connection is a polyline through the route ELK computed, with an arrowhead, and its name where ELK placed its label:

Listing 9. A connection

```
private void connection(Connection connection, Container parent, double x, double y) {
    if (connection.getWaypoints().size() < 2) {
        return;
    }
    StringBuilder points = new StringBuilder();
    for (Point point : connection.getWaypoints()) {
        points.append(number(x + point.getX())).append(',').append(number(y + point.getY
())).append(' ');
    }
    Polyline polyline = parent.add(factory.createPolyline());
    polyline.setId(connection.getId());
    polyline.setPoints(points.toString().trim());
    polyline.setFill("none");
    polyline.setStroke("#333333");
    polyline.setMarkerEnd("url(#arrow)"); ①
    if (connection.getAttributes().containsKey("async")) {
        polyline.setStrokeDasharray("6 4");
    }
    for (DiagramElement child : connection.getChildren()) {
        if (child instanceof Label label && label.getBounds() != null) {
            Bounds b = label.getBounds(); ②
            Text text = parent.add(factory.createText());
            text.setX(number(x + b.getX() + b.getWidth() / 2));
            text.setY(number(y + b.getY() + b.getHeight() / 2 + FONT_SIZE * 0.35));
            text.setTextAnchor("middle");
            text.setFill("#333333");
            text.addText(label.getText() == null ? connection.getName() : label.getText());
        }
    }
}
```

① The arrowhead is a marker, defined once in `defs` and referred to by every line.

② ELK reserved this rectangle for the label, so names do not overlap lines or boxes.

Part III: Looking Ahead

What comes next for generated diagrams.

Chapter 7. Looking ahead

Everything in this chapter is **PLANNED**.

7.1. A diagram to SVG module

The converter of Chapter 6 as a module, `models/diagram/svg`, with what a converter for real diagrams needs: shapes by a plug-in point rather than a hard-coded rectangle, labels of several lines, ports, nested nodes drawn as frames, and styles from attributes. Layout stays in the ELK module, so ELK's dependencies stay out of the path from a laid-out diagram to SVG.

7.2. Measured text

A table of glyph widths for the font the books use, Noto Sans, shipped as a resource of the diagram model, and a text measurer that reads it: accurate for that font, the same on every machine, and with no dependency on installed fonts. The SVG converter then names the font it measured.

7.3. Class context diagrams

The classes of an Ecore or Meta model, one diagram per class: the class, the classes it extends and that extend it, and the classes it refers to and that refer to it, generated for every class of every model in the runtime's documentation, each box a link to the class's page with its documentation as a tooltip.

7.4. Dark mode

Presentation attributes for the PDF, which prawn-svg reads reliably, and, for the HTML, CSS classes with a block for dark color schemes.

7.5. PNG

For platforms that do not take SVG, such as Medium and LinkedIn, the SVG rendered as PNG in a browser, through the runtime's Playwright module.

7.6. Reading other tools' SVG losslessly

Elements and attributes outside the subset kept when SVG is read, and written back, so that SVG from PlantUML or Draw.io can be post-processed: links added to a PlantUML diagram, or colors changed in a Draw.io export.

Appendix A: Reference

A.1. Java classes

Class	What it is for
`resource.SvgFormat`	`save(svg, out)` and `toString(svg)` write a copy of an `svg` element as SVG; `load(in, uri)` reads one.
`resource.SvgResourceContentsHandler`	The contents handler of `.svg` resources.
`services.SvgCapabilityFactory`	Offers the contents handler for the last qualifier, `svg`.
`services.SvgResourceSetContributor`	Registers the resource factory for `svg`, unless one is registered.
`services.SvgEPackageProvider`	Provides the package as a service.

A.2. Writing options

`SvgFormat` writes and reads with EMF's XML resource and these options:

Option	Why
`OPTION_EXTENDED_META_DATA`, with the SVG package	Element and attribute names are SVG's
`OPTION_ENCODING`: UTF-8	
`OPTION_LAX_FEATURE_PROCESSING`	Elements and attributes outside the subset are skipped on read, not errors
`OPTION_USE_LEXICAL_HANDLER`	Character data is read as it is
`OPTION_SAVE_TYPE_INFORMATION`: false	No `xsi:type`: each element's name says its class
`OPTION_FORMATTED`, `OPTION_LINE_WIDTH`	Indented output that reads well in a diff
No external DTDs	SVG 1.1's DTD is not downloaded on read

Appendix B: Building this book, and its figure

This book is AsciiDoc in `models/svg/model/src/docs/asciidoc`, built with the model after its tests have passed:

```
mvn -pl models/svg/model -am verify                          # the model, its tests and this book
mvn -pl models/svg/model verify -Dzephyr.editor-notes=false   # without editor's notes
mvn -pl models/svg/model verify -Dasciidoctor.skip=true       # without the book
```

The PDF and the single-page HTML are written to `target/book` and attached to the model's artifact with the classifier `book`; the test sources are attached with the classifier `test-sources`. The look comes from the runtime's `doc/zephyr` directory, shared by all the Nasdanika Zephyr Books.

Listings are tagged regions of the tests in `src/test/java/org/nasdanika/sdk/runtime/models/svg/tests`. A listing line holds at most 96 characters, and a warning fails the build.

B.1. The figure

The figure, `internet-banking.svg` in `src/test/resources/org/nasdanika/sdk/runtime/models/svg/tests`, is the file that `SvgTests.testDiagram` writes to `target/svg`, copied. To change it, change the test or the converter, run the tests, and copy the file again.

The figure is also the test of the subset against Asciidoctor PDF's SVG renderer: a warning from the renderer fails the build of the PDF.

Appendix C: Glossary

ExtendedMetaData

EMF's annotations that map a model to XML: which element or attribute a class or a feature is, in which namespace. EMF puts them on models it generates from XML schemas; this model has them by hand.

Feature map

An EMF list of entries, each a feature and a value, which keeps values of different features in one order. Here, the children of a container and the characters and spans of a text.

Layout

Positions and sizes of nodes, routes of connections and positions of labels, computed here by the Eclipse Layout Kernel.

Marker

A small drawing at the vertices of a line, such as an arrowhead, defined once and referred to by the lines that use it.

Presentation attribute

An SVG attribute that says how an element is painted, such as `fill` or `stroke-width`. Inherited by an element's children.

prawn-svg

The library Asciidoctor PDF draws SVG with. What it draws sets the subset of this model.

Qualifier

A part of a file name between dots. In `system.drawio.svg`, the qualifiers are `svg`, `drawio` and `system`, read right to left.

View box

The rectangle of user coordinates that an `svg` element shows, `min-x min-y width height`.